

INTERNAL PRODUCT BLUEPRINT

Voice Agent OS Platform Blueprint

Operating platform for new voice-agent builds, live-client optimisation and Elk prompt/function production.

PREPARED FOR

Inspra AI

VERSION

v1

DATE

27 June 2026

SOURCE MATERIAL

Main platform blueprint plus Elk sub-blueprint

Purpose

This PDF turns the two markdown blueprints into a diagram-led operating blueprint. It shows how the main Voice Agent OS platform and the Elk Prompt and Function Builder layer fit together, what should be built first, and which decisions remain open before implementation.

1. Blueprint Summary

Voice Agent OS should be built as an internal operating platform for the full client lifecycle, not only as a one-time prompt generator. The main platform handles intake, client workspaces, monitoring, work item triage, releases and wiki learning. The build package must produce five required artefacts every time: blueprint, sales script, call flow chart, agent prompt and integration flowchart. The Elk Prompt and Function Builder is the production layer that creates, revises, validates and exports prompt/function packages when the main platform decides a build or revision is needed.

Operating mode	What it does
New client onboarding	Turns discovery material into a production-ready voice agent with prompt, config, QA, approval and monitoring.
Existing client optimisation	Turns emails, meetings, call analysis and monitor findings into tracked work items, safe fixes, recommendations and releases.
Elk Prompt and Function Builder	Sub-blueprint layer that standardises prompt generation, function packages, QA checks and Elk-ready exports.

2. Master Operating Model

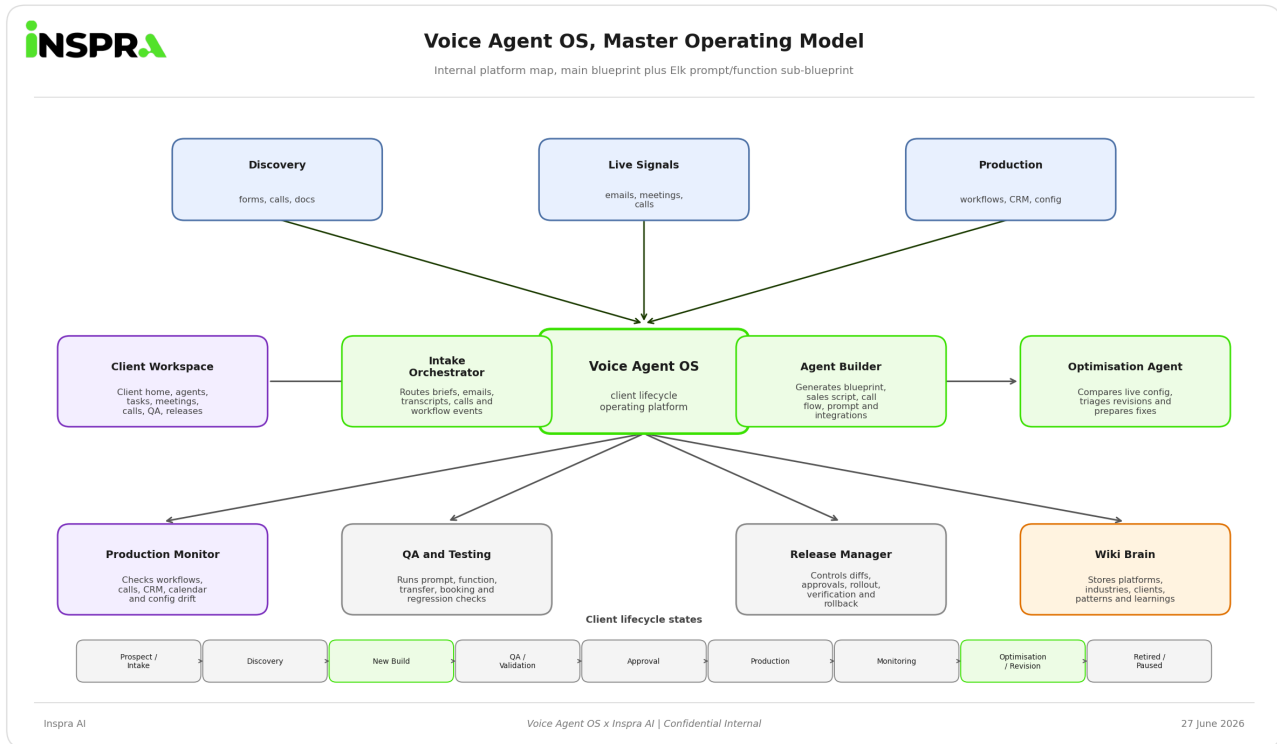


Figure 1: Voice Agent OS master operating model.

The platform has a single client workspace at the centre. Intake routes new material into either the new-build lane or the recurring optimisation lane. QA, release control, monitoring and the wiki brain sit across both lanes so that every production edit is traceable and every useful lesson can be reused.

3. Client Lifecycle and Workspace

Each client should move through a clear lifecycle, from prospect/intake through discovery, build, approval, production, monitoring and optimisation. The workspace is the durable record that connects stakeholders, agents, configs, meetings, emails, calls, releases, QA, decisions and learnings.

Lifecycle states	Prospect / Intake, Discovery, New Build, QA / Validation, Approval, Production, Monitoring, Optimisation / Revision, Retired / Paused
Workspace area	Included records
Workspace	Client profile, Contacts and stakeholders, Voice agents, Platform metadata, Fathom meetings, Email threads, Production prompt/config, Prior versions
Operations	Function/tool configs, Deployment history, QA reports, Call logs, Open issues, Monitoring rules, Decisions and approvals, Wiki learnings

4. New Client Onboarding Workflow

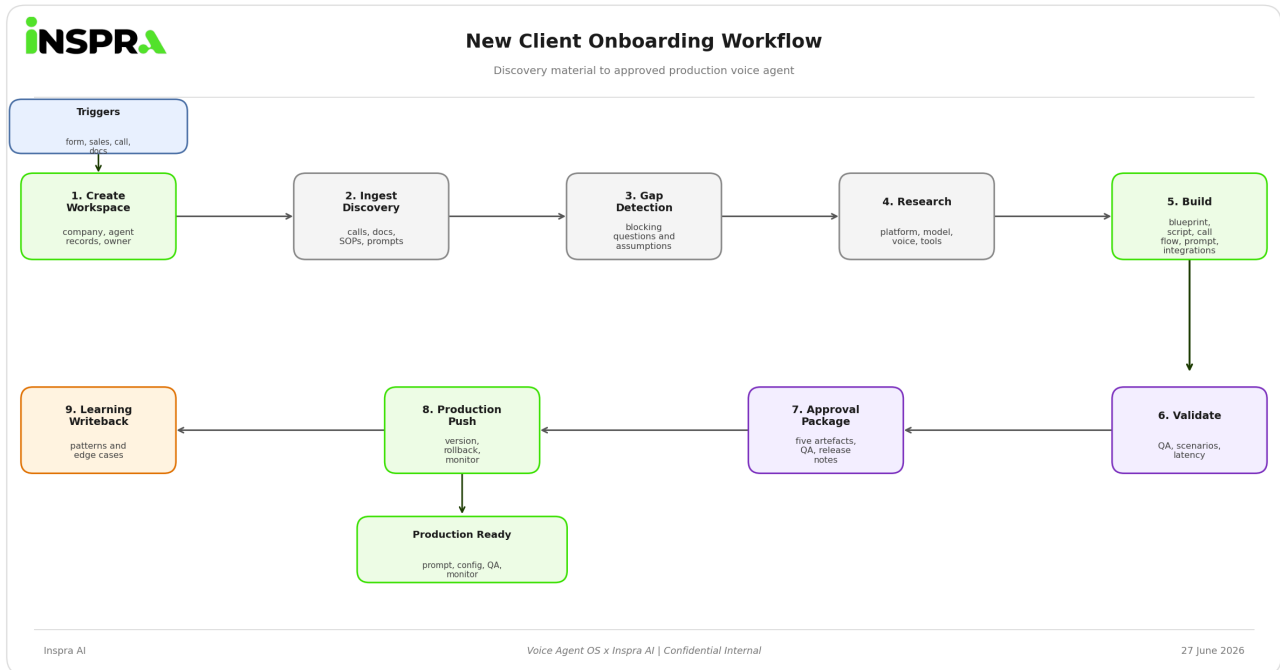


Figure 2: New client onboarding workflow.

This lane starts when a client brief, sales handoff, discovery call, transcript, uploaded document or existing prompt arrives. The workflow creates a workspace, ingests source material, detects gaps, researches platform and voice choices, builds prompt/config/function assets, validates them, packages approval material, pushes production only after approval and writes reusable lessons back to the wiki.

5. Existing Client Optimisation Workflow

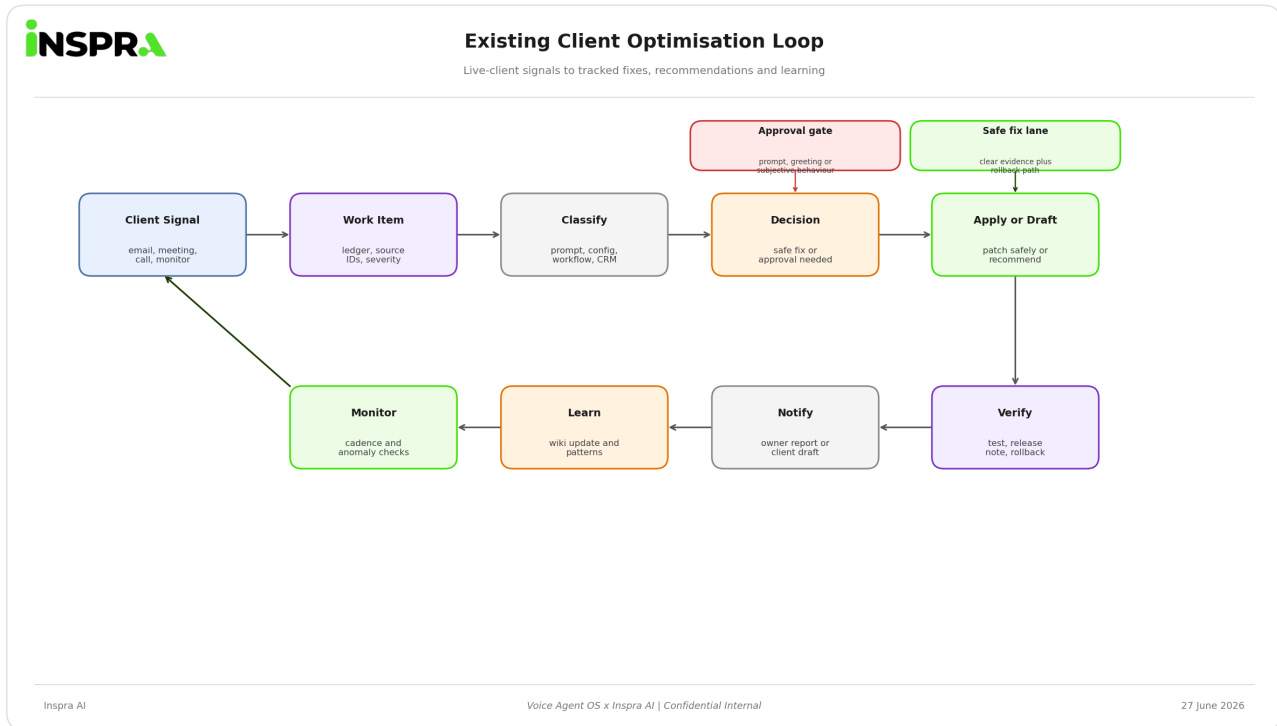


Figure 3: Existing client optimisation loop.

This lane handles live clients after launch. Signals from emails, Fathom meetings, call logs, monitors, workflows, CRM and calendar systems become work items. Each item is classified, checked for safe-fix eligibility, routed to either a controlled operational fix or an approval recommendation, verified, reported and fed back into monitoring and the wiki.

6. Revision Triage Model

The work item ledger should preserve source IDs, thread IDs, stakeholder, severity, owner, safe-to-fix flag, approval flags, evidence, proposed fix, applied fix, verification result and rollback details. The triage model keeps subjective voice behaviour changes away from direct production edits while allowing safe operational repairs to move quickly.

Bucket	Default action	Blueprint rule
Prompt or greeting	Approval required	Draft exact suggested wording and rationale before production changes.
Agent non-prompt config	Safe fix when clear	Patch function descriptions, transfer settings or knowledge entries with verification.
Workflow automation	Safe fix when clear	Fix routing, webhook, notification or scheduler logic, then report.
CRM or data issue	Controlled test only	Snapshot before testing, revert mutations, prove writeback before marking complete.
Calendar or booking	Verify proof	Check routing, calendar event proof and CRM consistency before booking claims.
Live call or transfer	Ask if unclear	Separate voice platform, telephony and automation ownership before changing routing.
Platform or API	Alert and patch	Handle outage, schema, token or webhook errors with owner notification and wiki learning.

7. Elk Prompt and Function Builder Sub-Blueprint

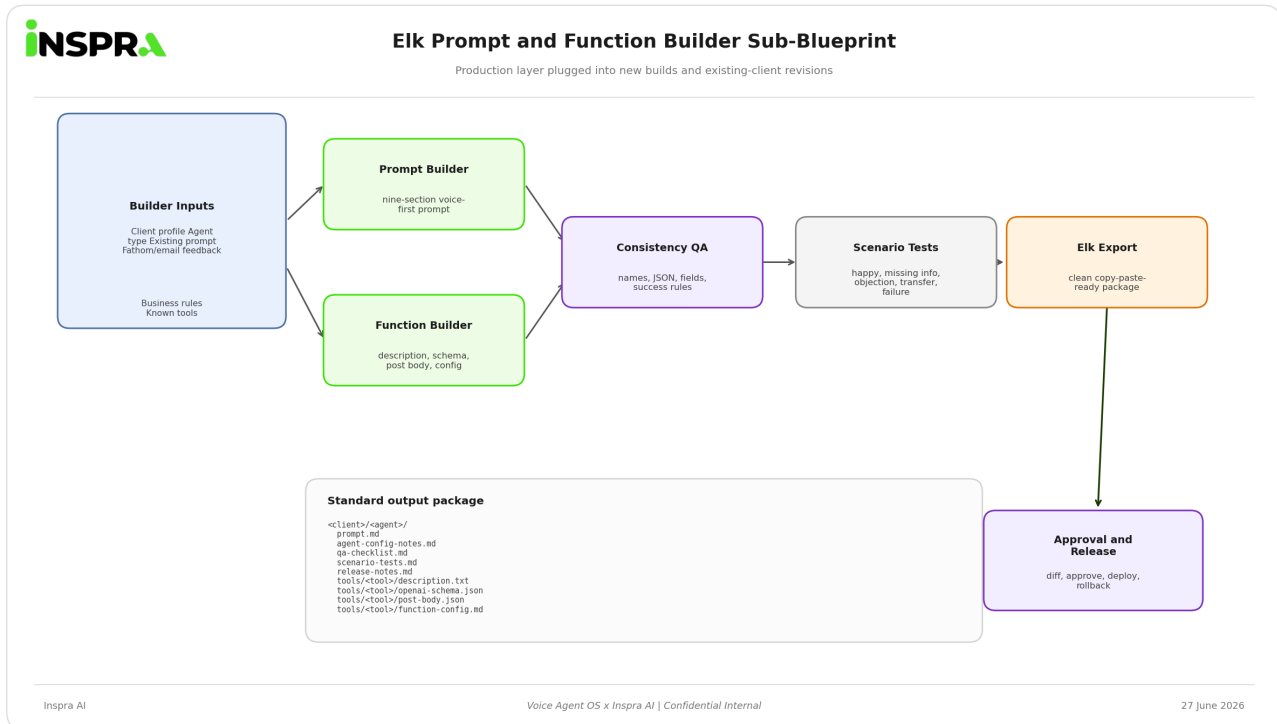


Figure 4: Prompt and function builder production layer.

The sub-blueprint plugs into both main workflows. The main platform decides when a prompt or function build is needed. The sub-system decides how the prompt/function package is produced, checked, exported and versioned.

8. Required Build Outputs

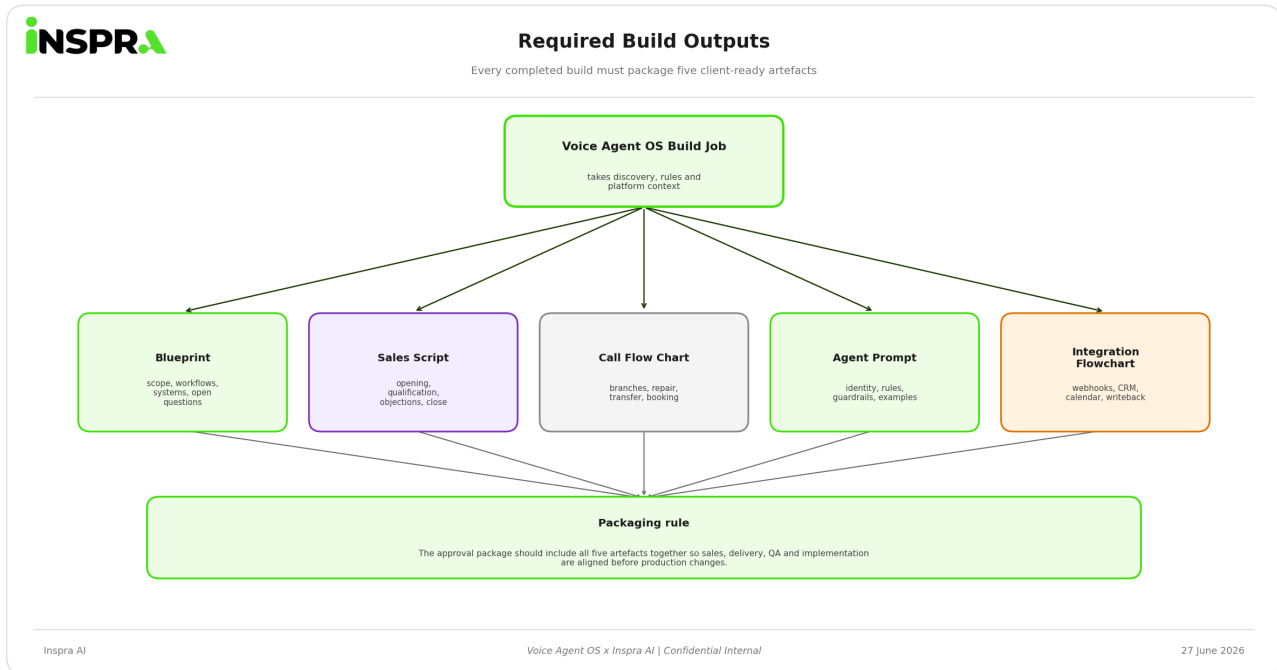


Figure 5: Required build outputs for every completed build.

Every build should package the same five artefacts so the sales view, call design, prompt implementation and integration plan stay aligned. These outputs are the minimum completion standard before approval or production handoff.

Required artefact	Purpose
Blueprint	Strategic operating blueprint covering scope, workflows, systems, open questions and implementation path.
Sales Script	Caller-facing sales or discovery script, including opening, qualification, objections, value framing and close.
Call Flow Chart	Voice conversation path covering opening, intent, slot collection, branches, transfer, booking, repair and closing.
Agent Prompt	Production-ready voice-agent prompt with identity, response rules, guardrails, workflow, tool rules and examples.
Integration Flowchart	System-to-system flow showing voice platform, automation, CRM, calendar, notifications, webhooks and data writeback.

9. Prompt and Function Production Standards

Prompt structure	Sections
Standard nine-section prompt	Role and Objective, Personality and Tone, Context, Instructions, Guardrails, Stages / Call Flow, Example Interactions, Knowledge Base, Voice Setup Checklist
Package file	Purpose
description.txt	Short tool description, when to use it, confirmation rules, success and failure handling.
openai-schema.json	Plain JSON request-body schema with required fields and strong field descriptions.
post-body.json	Webhook payload mapping using collected values and runtime variables.
function-config.md	Label, name, event key, method, endpoint, headers, response, success condition and testing notes.

10. Function Categories and QA Gates

Category	Common functions	Rule
Calendar	check availability, book appointment	Check availability first, present slots, confirm chosen slot, only confirm after function success.
CRM / HubSpot	contact, update, task, outcome	Avoid uncontrolled test mutations and keep writeback proof visible in QA.
Notification	SMS, email, internal alert	Do not treat a notification as proof of booking or a live transfer.
Transfer	warm transfer, cold transfer	Separate live handoff from cold-transfer fallback and avoid duplicate handoff scripts.
End call	end_call	Let the platform handle the final message when supported.
Callback / Support	callback, route, notify, outcome	Model callback flow separately from cold outbound and ask when routing ownership is unclear.

QA gates

- Prompt uses the standard nine-section structure.
- Function names are exact and consistent across prompt, schema and config.
- Required fields are collected before function calls.
- The agent confirms success only after the function succeeds.
- Function names are never spoken to callers.
- Schemas and post bodies are valid JSON.
- Calendar functions are sequenced correctly.
- Warm transfer, cold transfer and notification-only flows are separated.
- Prompt or greeting changes are separated from safe operational config fixes.
- Scenario tests cover happy path, missing information, objections, transfer, off-topic, function failure and booking confirmation.

11. Information Architecture

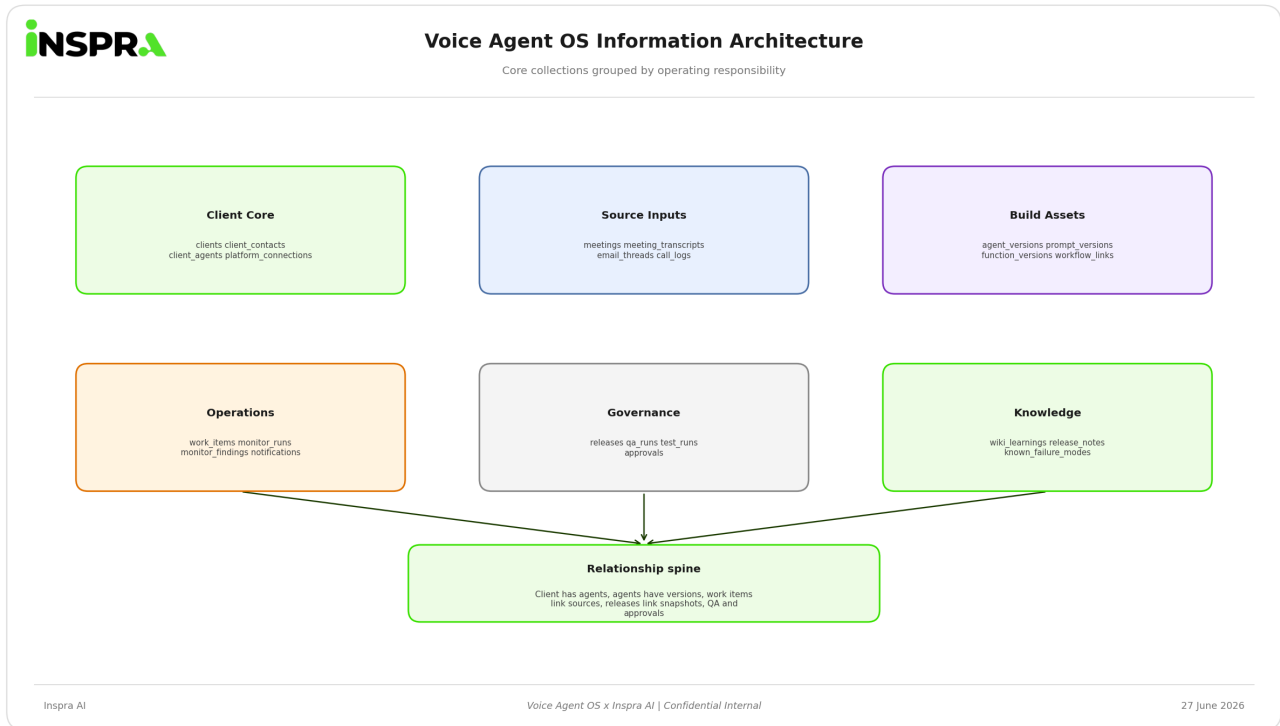
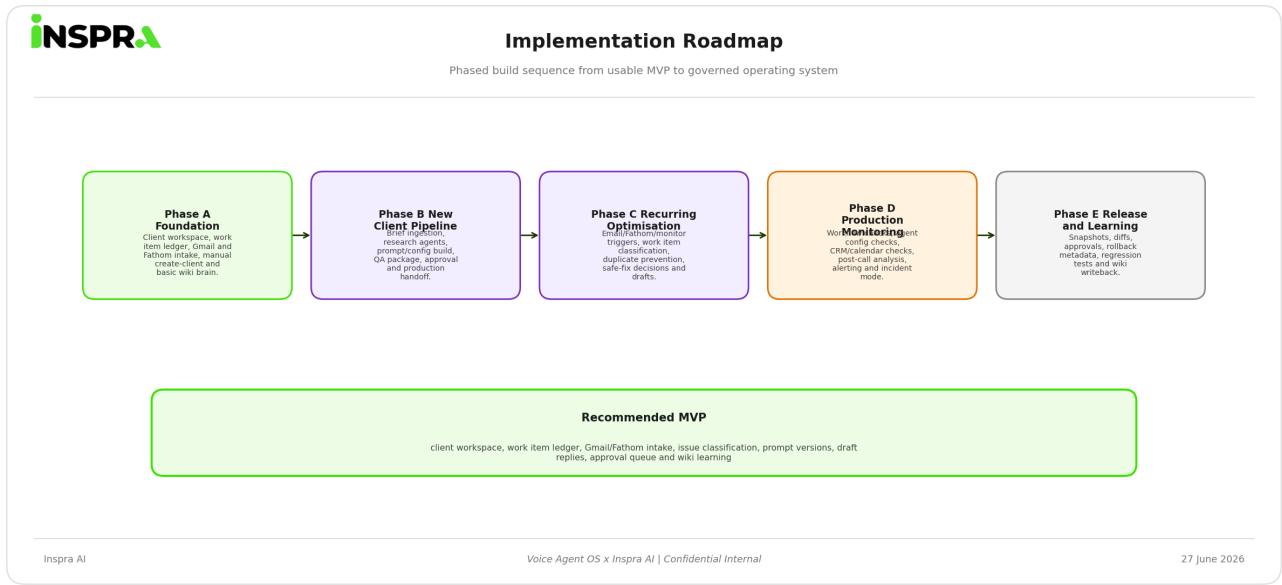


Figure 6: Information architecture grouped by operating responsibility.

The recommended data model centres on clients, agents, work items, releases and learnings. Work items should be able to link back to email, Fathom, call, workflow, CRM and release records. Releases should link to before/after snapshots, QA runs and approval records.

12. MVP and Implementation Roadmap



14. Open Questions Before Build

- Fathom access method: API, webhook, summary email, or manual transcript upload?
- Source of truth for client records: platform database, HubSpot, Airtable, Notion, or existing files?
- Production control level: safe auto-fixes or approval queue first?
- Client communication rule: draft-only replies or approved automatic status emails?
- First platforms to support: Inspira, Elk, Vapi, Retell, LiveKit, Genius, n8n, HubSpot, Dialpad?
- Monitoring cadence: thirty minutes for all active clients or only high-touch pilots?
- Approval matrix: who approves prompt changes, config changes, workflow changes and production releases?

Recommended next step: confirm the source-of-truth database, Fathom access path, approval matrix and auto-fix policy first. Those decisions determine the platform permissions, data model and release controls.

15. Build Notes for the Product Team

Treat prompt/greeting changes as controlled releases. Treat non-prompt config, workflow, CRM and calendar fixes as safe only when there is clear evidence, a snapshot, a rollback path and a verification result. Keep the five required build artefacts together: blueprint, sales script, call flow chart, agent prompt and integration flowchart. Keep prompt text separate from tool schemas and post bodies. Keep Elk exports copy-paste-ready rather than turning them into long implementation notes.

The sub-blueprint should be implemented as a reusable production service, not as a one-off wizard. It should be callable from new-build flows, revision flows, Fathom intake, Gmail intake, the production monitor, QA/test engine, release manager and wiki learning loop.